Project Proposal: Protocol Police

Extensible typestate verification for MiniScala

Filippo Nardon, Giovanni Maria Zanchetta

November 4, 2026

Contents

Idea **1**

Motivation and research basis **1**

Analysis goals **2**

MiniScala adaptations **2**

Milestones and timeline **3**

Evaluation **3**

Process, AI use and testing **3**

References **4**

Idea

Many APIs are only correct when their operations are called in a particular order: a file must be opened before it is read and closed before the program ends; a transaction must begin before it is used and must end in a commit or a rollback; a connection must be established before data is sent. These *usage protocols* are usually documented informally, and violating them is a common source of bugs.

Protocol Police is a static analyzer for MiniScala that verifies user programs against such protocols. Library developers declare a protocol directly on a class through annotations: the constructor declares the initial state and the set of final states, and each method declares the state transitions it causes. The analyzer then checks the *user program* that uses the class, never the class implementation itself. New protocols are introduced by writing new annotated classes, without modifying the analysis engine.

This is typestate analysis [Strom and Yemini 1986]. Our contribution is not a new algorithm but the adaptation and integration of established techniques into a reusable verifier for MiniScala, together with an empirical evaluation of how well it works.

Motivation and research basis

Typestate was introduced by Strom and Yemini [1986] and has since been studied both as a type discipline and as a whole-program analysis. We draw on several lines of work:

- **Protocols declared on classes.** Fugue [DeLine and Fähndrich 2004] and Plural [Bierhoff and Aldrich 2007] let developers annotate object methods with typestate pre- and postconditions, and the Checker Framework's Resource Leak Checker [Kellogg et al. 2021] verifies that resources are closed using library annotations. Our annotation design follows this tradition, but we check the client program with a whole-program analysis instead of modular type checking, so that clients need no annotations of their own.

- **Extensible checkers.** Hallem et al. [2002] separate protocol-specific state machines from a reusable analysis engine. We adopt this separation, but unlike their bug-finding system, which explicitly accepts missed bugs, we aim for conservative verification.
- **Path sensitivity.** ESP [Das et al. 2002] shows that tracking which branch outcomes correlate with which protocol states removes many false alarms, and that this can be done efficiently for file-like protocols.
- **Aliasing.** Fink et al. [2008] show that typestate verification depends on precise alias information: a state change can only be applied as a strong update when the analysis knows which object is affected.

We build the analysis on the course material [Møller and Schwartzbach] and the classic work behind it: monotone frameworks, interprocedural analysis with call-string or functional contexts [Sharir and Pnueli 1981; Reps et al. 1995], inclusion-based points-to analysis [Andersen 1994] and control-flow analysis [Shivers 1991]. SPA presents a single fixed open/close protocol as an example; we generalize this to many objects and user-declared protocols, across function calls.

At its core, Protocol Police combines **typestate analysis, control-flow analysis, pointer analysis, context sensitivity and path sensitivity**. These are a starting point rather than a fixed recipe: we expect to add supporting analyses where the protocols need them, for example a simple value analysis to decide the return-value guards described below.

Analysis goals

Our goal is **conservative verification of protocol usage**: we check every use of an annotated class against the protocol its annotations declare. For the supported MiniScala subset, we report one of three outcomes for each protocol requirement:

- **Verified:** the requirement holds on every execution.
- **Warning:** the analysis cannot exclude a violation (for example, a file may be either `Open` or `Closed` at a `read()`).
- **Error:** a violation is established.

We check two kinds of requirements: every method call must happen in an allowed state, and at the end of the program every protocol object must be in one of its declared final states. We define completion on the *current* state, so reopening a closed file renews the obligation to close it.

We also let annotations relate a method's return value to the resulting state, using an ordered case block. For example, a commit operation that returns a status code:

```
@protocol(init = Idle, final = {Idle, Committed})
class Transaction() {
  @from(Idle) @to { case _ => Active }
  def begin(): Unit = ()

  @from(Active) @to { case 0 => Committed; case n if n > 0 => Active; case _ => Failed }
  def commit(): Int = 0
}
```

When the result is unknown, we keep every outcome that cannot be ruled out, together with the correlation between the result and the state, so that a later test such as `if (status == 0)` can refine it. This is where path sensitivity pays off. We allow a method to carry several transition declarations (for example, closing two files passed as arguments), and we treat unannotated methods as leaving protocol states unchanged. When an annotated method returns a protocol object, we make no assumption about which object it is unless an annotation declares it.

MiniScala adaptations

We reuse the supplied parser, type checker, interpreter, intermediate representation and CFG construction. We make three additions to the language:

1. **Annotations.** We extend the lexer, parser and AST with annotations on class declarations (initial and final states) and on method declarations (transitions). We extend the type checker to validate state names and guard expressions, and let the interpreter ignore annotations.

2. **Imports.** MiniScala programs consist of a single file. We add a minimal `import "path"` directive: we restrict imported files to class declarations, load each file once, reject cyclic imports, and add the imported classes to the program's outer block before type checking. We extend source positions with a file name so that diagnostics point to the right file.
3. **Stub libraries.** MiniScala has no files, transactions or networking, so we write *stub* classes whose bodies only fake the behaviour, for example a `read()` that returns a constant. We consider this sufficient, since our analysis concerns how an API is used, not how it is implemented. We put each protocol in its own importable library file.

```
import "lib/file.s"
{
  val f = new File("data.txt");
  f.open();
  f.read()          // error: f definitely ends Open, which is not a final state
}
```

We start with a core subset (values, local variables, objects and references, annotated calls and conditionals) and add loops, user-defined functions and aliasing next. Higher-order functions and closures, in particular closures capturing mutable variables, are expansion goals.

Milestones and timeline

We estimate a combined effort of roughly 60–80 hours. The milestones below are indicative: each one should leave a working, tested analyzer, and later scope is adjusted to actual progress.

Period	Milestone
Nov 6–12	Annotation syntax, imports, file-aware positions and the `File` stub. Intraprocedural typestate check on straight-line code and conditionals, including the final-state check, with tests.
Nov 13–19	Pointer analysis for aliases, loops, function calls with call resolution, and context sensitivity.
Nov 20–26	Path-sensitive refinements (return values correlated with states), transactions as a second protocol, broader language coverage.
Nov 27–Dec 4	Evaluation and fixes; presentation on December 3 and report on December 4.

Evaluation

We will build a benchmark suite alongside the analyzer, using protocols inspired by well-known APIs: files (Java `Closeable`), transactions (SQL `BEGIN`/`COMMIT`/`ROLLBACK`) and, if time allows, connections (Java `Socket`). We deliberately keep these as small MiniScala models; they do not reproduce the real APIs in full. The suite contains small controlled programs that isolate branches, loops, aliases, calls and return-value tests, plus a few larger programs that combine features. We will measure:

- **Extensibility:** adding protocols only through annotations, with no engine changes.
- **Verification quality:** safe programs verified, violations confirmed, warnings on safe programs, and any invalid program wrongly verified.
- **Cost:** analysis time, and possibly memory, as program size and features grow.

Where practical, we will compare configurations with and without context and path sensitivity to show what the extra precision buys and what it costs.

Process, AI use and testing

We will follow an agile process supported by BMad: short iterations deliver small, testable increments from a prioritized backlog, and we review progress and results regularly to adjust scope. We will work by **pair programming**, rotating

driver and navigator roles, so that both of us review every design decision and every piece of AI-generated code as it is written.

We will use AI, through the BMad workflow, for literature exploration, explaining algorithms, implementation assistance, and generating test variants. Generated code is reviewed by the pair and validated against expected results that we establish ourselves. We will keep short notes on useful assistance, mistakes and corrections for the final reflection.

Testing grows with the implementation:

- **Annotation checks:** malformed or inconsistent protocol declarations are rejected.
- **Expected-outcome tests:** small programs with manually established results covering verified, warning and error cases.
- **Feature combinations:** branches, loops, aliases and calls added progressively, then combined.
- **Regression tests:** earlier examples are kept as the supported subset grows.

A passing test suite is evidence of correctness, not a soundness proof.

References

- L. O. Andersen. *Program Analysis and Specialization for the C Programming Language*. PhD thesis, DIKU, University of Copenhagen, 1994.
- K. Bierhoff and J. Aldrich. Modular typestate checking of aliased objects. *OOPSLA*, 2007.
- M. Das, S. Lerner and M. Seigle. ESP: Path-sensitive program verification in polynomial time. *PLDI*, 2002.
- R. DeLine and M. Fähndrich. Typestates for objects. *ECOOP*, 2004.
- S. J. Fink, E. Yahav, N. Dor, G. Ramalingam and E. Geay. Effective typestate verification in the presence of aliasing. *ACM TOSEM*, 2008.
- S. Hallem, B. Chelf, Y. Xie and D. Engler. A system and language for building system-specific, static analyses. *PLDI*, 2002.
- M. Kellogg, N. Shadab, M. Sridharan and M. D. Ernst. Lightweight and modular resource leak verification. *ESEC/FSE*, 2021.
- A. Møller and M. I. Schwartzbach. *Static Program Analysis*. Aarhus University.
- T. Reps, S. Horwitz and M. Sagiv. Precise interprocedural dataflow analysis via graph reachability. *POPL*, 1995.
- M. Sharir and A. Pnueli. Two approaches to interprocedural data flow analysis. In *Program Flow Analysis: Theory and Applications*, 1981.
- O. Shivers. *Control-Flow Analysis of Higher-Order Languages*. PhD thesis, Carnegie Mellon University, 1991.
- R. E. Strom and S. Yemini. Typestate: A programming language concept for enhancing software reliability. *IEEE TSE*, 1986.